

How to Use Wireshark (Beginner-Friendly Guide)

1□ Installing Wireshark

- Download from the official site (Windows / Linux / macOS).
 - On **Windows**, install **Npcap** when prompted (required for packet capture).
 - On **Linux**, you may need root or add your user to the `wireshark` group.
-

2□ Understanding the Wireshark Interface

Wireshark has **3 main panes**:

1. **Packet List Pane (Top)**
 - Shows captured packets
 - Columns: Time, Source, Destination, Protocol, Length, Info
 2. **Packet Details Pane (Middle)**
 - Expands protocol layers (Ethernet, IP, TCP, HTTP, etc.)
 3. **Packet Bytes Pane (Bottom)**
 - Raw packet data in Hex + ASCII
-

3□ Starting a Packet Capture

Steps

1. Open Wireshark
2. Select a **network interface**:
 - Ethernet → Wired connection
 - Wi-Fi → Wireless connection
3. Click **Start (blue shark fin)**

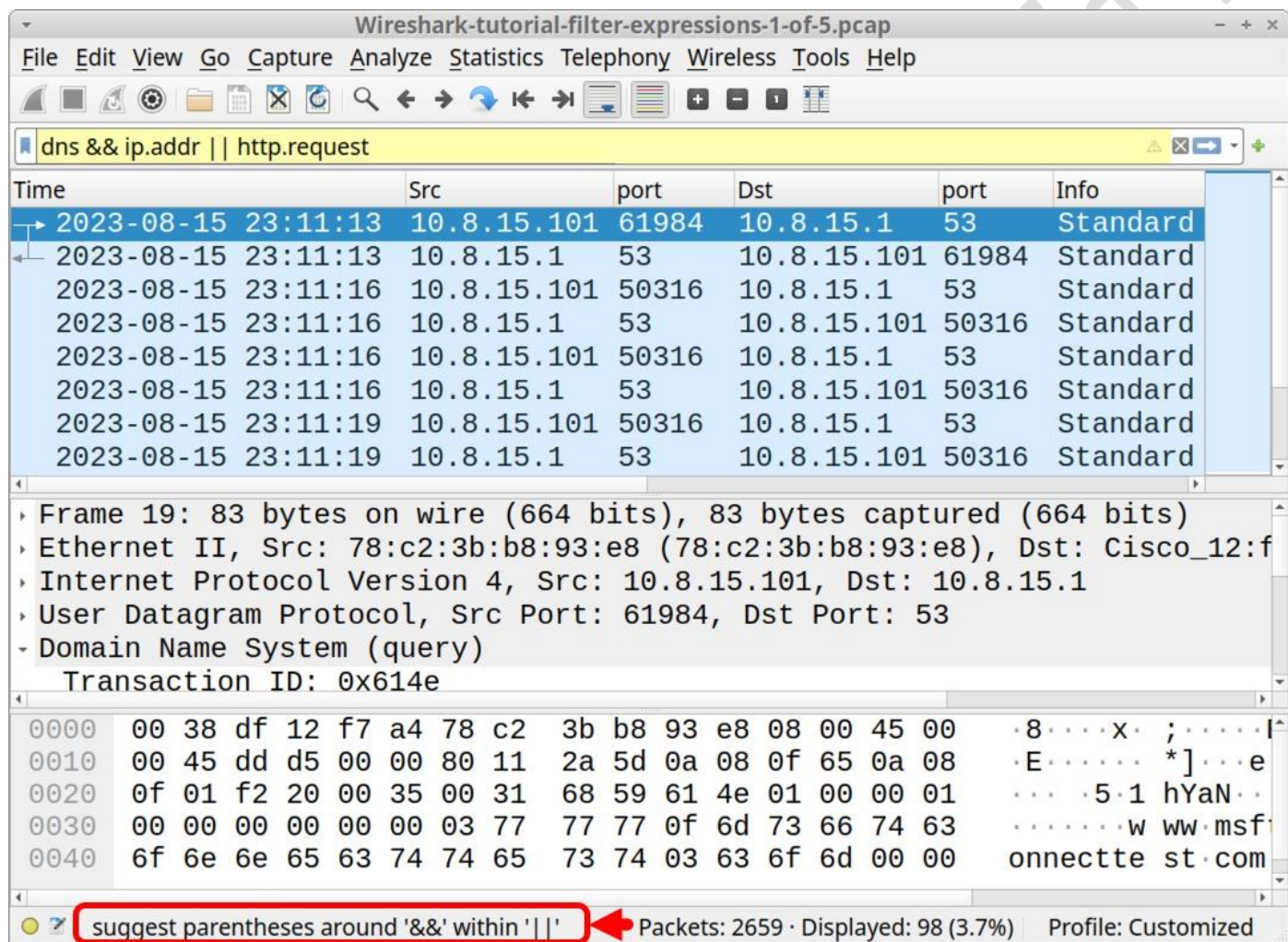
★ Tip: Choose the interface showing **live traffic activity**

4 □ Stopping & Saving Captures

- Click **Stop (red square)** to stop capturing
- Save file as .pcap or .pcapng
- Useful for **forensic analysis or reports**

5 🔍 Useful Display Filters (Simple & Easy)

Display filters are used **after capturing packets** to show only what you want. They work the **same in Wireshark and TShark**.



The image shows the Wireshark interface with a display filter applied. The filter bar at the top contains the text `dns && ip.addr || http.request`. Below the filter bar, a list of captured packets is displayed. The first packet is selected, and its details are shown in the packet details pane. The packet details pane shows the following information:

- Frame 19: 83 bytes on wire (664 bits), 83 bytes captured (664 bits)
- Ethernet II, Src: 78:c2:3b:b8:93:e8 (78:c2:3b:b8:93:e8), Dst: Cisco_12:f
- Internet Protocol Version 4, Src: 10.8.15.101, Dst: 10.8.15.1
- User Datagram Protocol, Src Port: 61984, Dst Port: 53
- Domain Name System (query)
 - Transaction ID: 0x614e

The packet list pane shows the following data:

Time	Src	port	Dst	port	Info
2023-08-15 23:11:13	10.8.15.101	61984	10.8.15.1	53	Standard
2023-08-15 23:11:13	10.8.15.1	53	10.8.15.101	61984	Standard
2023-08-15 23:11:16	10.8.15.101	50316	10.8.15.1	53	Standard
2023-08-15 23:11:16	10.8.15.1	53	10.8.15.101	50316	Standard
2023-08-15 23:11:16	10.8.15.101	50316	10.8.15.1	53	Standard
2023-08-15 23:11:16	10.8.15.1	53	10.8.15.101	50316	Standard
2023-08-15 23:11:19	10.8.15.101	50316	10.8.15.1	53	Standard
2023-08-15 23:11:19	10.8.15.1	53	10.8.15.101	50316	Standard

At the bottom of the interface, a status bar shows the following information:

- suggest parentheses around '&&' within '||'
- Packets: 2659 · Displayed: 98 (3.7%)
- Profile: Customized

test.pcap - Wireshark

File Edit View Go Capture Analyze Statistics Help

Filter: tcp Expression... Clear Apply

No.	Time	Source	Destination	Protocol	Info
11	1.226156	192.168.0.2	192.168.0.1	TCP	3196 > http [SYN] Seq=0 Len=0 MSS
12	1.227282	192.168.0.1	192.168.0.2	TCP	http > 3196 [SYN, ACK] Seq=0 Ack=
13	1.227325	192.168.0.2	192.168.0.1	TCP	3196 > http [ACK] Seq=1 Ack=1 Win
14	1.227451	192.168.0.2	192.168.0.1	HTTP	SUBSCRIBE /upnp/service/Layer3For
15	1.229309	192.168.0.1	192.168.0.2	TCP	http > 3196 [ACK] Seq=1 Ack=256 W
16	1.232421	192.168.0.1	192.168.0.2	TCP	[TCP Window Update] http > 3196
17	1.248355	192.168.0.1	192.168.0.2	TCP	1025 > 5000 [SYN] Seq=0 Len=0 MSS
18	1.248391	192.168.0.2	192.168.0.1	TCP	5000 > 1025 [SYN, ACK] Seq=0 Ack=
19	1.250171	192.168.0.1	192.168.0.2	HTTP	HTTP/1.0 200 OK
20	1.250285	192.168.0.2	192.168.0.1	TCP	3196 > http [FIN, ACK] Seq=256 Ac
21	1.250810	192.168.0.1	192.168.0.2	TCP	http > 3196 [FIN, ACK] Seq=114 Ac
22	1.250842	192.168.0.2	192.168.0.1	TCP	3196 > http [ACK] Seq=257 Ack=115
23	1.251868	192.168.0.1	192.168.0.2	TCP	1025 > 5000 [ACK] Seq=1 Ack=1 Win
24	1.252826	192.168.0.1	192.168.0.2	TCP	http > 3196 [FIN, ACK] Seq=26611
25	1.253323	192.168.0.2	192.168.0.1	TCP	3197 > http [SYN] Seq=0 Len=0 MSS
26	1.254502	192.168.0.1	192.168.0.2	TCP	http > 3197 [SYN, ACK] Seq=0 Ack=
27	1.254532	192.168.0.2	192.168.0.1	TCP	3197 > http [ACK] Seq=1 Ack=1 Win

Frame 11 (62 bytes on wire, 62 bytes captured)

Ethernet II, Src: 192.168.0.2 (00:0b:5d:20:cd:02), Dst: Netgear_2d:75:9a (00:09:5b:2d:75:9a)

Internet Protocol, Src: 192.168.0.2 (192.168.0.2), Dst: 192.168.0.1 (192.168.0.1)

Transmission Control Protocol, Src Port: 3196 (3196), Dst Port: http (80), Seq: 0, Len: 0

```

0000  00 09 5b 2d 75 9a 00 0b 5d 20 cd 02 08 00 45 00  ..[-u... ] ....E.
0010  00 30 18 48 40 00 80 06 61 2c c0 a8 00 02 c0 a8  .0.H@... a,.....
0020  00 01 0c 7c 00 50 3c 36 95 f8 00 00 00 00 70 02  ...|.P<6 .....p.
0030  fa f0 27 e0 00 00 02 04 05 b4 01 01 04 02      ...'.....

```

File: "D:\test.pcap" 14 KB 00:00:02 P: 120 D: 103 M: 0 [Expert: Error]

Wireshark-tutorial-filter-expressions-5-of-5.pcap

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

smtp.data.fragment basic basic+ basic+dns

Time	Dst	Dst port	Info
2023-03-16 15:33:33	27.121.3.241	587	from: =?UTF-8?B?PEXpc3RhIGRlIGRpc3RyYWJ1ac0nw6NvI
2023-03-16 15:33:34	27.121.3.241	587	from: "<Secretariado DSSA> seguranca.alimentar@dg
2023-03-16 15:33:35	27.121.3.241	587	from: =?UTF-8?B?PEXpc3RhIGRlIGRpc3RyYWJ1ac0nw6NvI
2023-03-16 15:33:36	27.121.3.241	587	from: "<Secretariado DSSA> seguranca.alimentar@dg
2023-03-16 15:33:37	27.121.3.241	587	from: =?UTF-8?B?PEXpc3RhIGRlIGRpc3RyYWJ1ac0nw6NvI
2023-03-16 15:33:38	27.121.3.241	587	from: "<Secretariado DSSA> seguranca.alimentar@dg
2023-03-16 15:33:39	27.121.3.241	587	from: =?UTF-8?B?PEXpc3RhIGRlIGRpc3RyYWJ1ac0nw6NvI

- Mark/Unmark Packet Ctrl+M
- Ignore/Unignore Packet Ctrl+D
- Set/Unset Time Reference Ctrl+T
- Time Shift... Ctrl+Shift+T
- Packet Comments
- Edit Resolved Name
- Apply as Filter
- Prepare as Filter
- Conversation Filter
- Colorize Conversation
- SCTP
- Follow
- Copy
- Protocol Preferences
- Decode As...
- Show Packet in New Window

TCP Stream Ctrl+Alt+Shift+T

UDP Stream Ctrl+Alt+Shift+U

DCCP Stream Ctrl+Alt+Shift+E

TLS Stream Ctrl+Alt+Shift+S

HTTP Stream Ctrl+Alt+Shift+H

HTTP/2 Stream

QUIC Stream

SIP Call

DATA fragmets: 6994 · Discarded: 7 (0.1%) Profile: Customized

Basic Display Filters (Most Used)

Display Filter	What it Shows	Simple Use
http	HTTP traffic	See website requests
https	HTTPS traffic	Encrypted web traffic
dns	DNS queries	See domain lookups
tcp	TCP packets	Reliable connections
udp	UDP packets	Fast connections (DNS, streaming)
icmp	Ping traffic	Check ping / connectivity

🌐 IP Address Filters

Filter	Use
<code>ip.addr == 192.168.1.10</code>	All traffic of this IP
<code>ip.src == 192.168.1.10</code>	Traffic coming from IP
<code>ip.dst == 192.168.1.20</code>	Traffic going to IP

★ Helpful in CTFs & troubleshooting

🔌 Port-Based Filters

Filter	Use
<code>tcp.port == 80</code>	HTTP traffic
<code>tcp.port == 22</code>	SSH traffic
<code>tcp.port == 21</code>	FTP traffic
<code>udp.port == 53</code>	DNS traffic

🌐 Website & Web Attacks (Pentesting)

Filter	Use
<code>http.request</code>	All HTTP requests
<code>http.response</code>	Server responses
<code>http.request.method == "GET"</code>	Normal page requests
<code>http.request.method == "POST"</code>	Login / form data

🔥 **POST filter is very important** for login analysis

Credential Hunting (HTTP only)

Filter	Use
<code>http.authorization</code>	Basic auth credentials
<code>frame contains "password"</code>	Find password text
<code>frame contains "username"</code>	Find usernames

✗ Will **NOT** work on HTTPS

CTF & Malware Analysis Filters

Filter	Use
<code>tcp.flags.syn == 1</code>	New connections
<code>tcp.flags.reset == 1</code>	Connection reset
<code>dns.qry.name contains "malware"</code>	Suspicious domains
<code>frame.len > 1000</code>	Large packets (exfiltration)

📁 File & Download Detection

Filter	Use
<code>http.response.code == 200</code>	Successful downloads
<code>http.response.code == 404</code>	File not found
<code>http.response.code == 500</code>	Server error

★ Beginner's Top 10 Filters (Must Remember)

```
http
dns
tcp
udp
icmp
ip.addr == x.x.x.x
tcp.port == 80
http.request.method == "POST"
frame contains "password"
tcp.flags.syn == 1
```

6 Wireshark in Cybersecurity & Pentesting

Wireshark helps in:

- Network reconnaissance
- Detecting suspicious traffic
- MITM attack analysis
- Malware communication analysis
- CTF challenges
- ✦ Often used with: Nmap, Netcat, Burp Suite, Metasploit

Logical Filters:

Logical filters are used to **combine multiple display filters** so you can be **more specific** about what packets you want to see.

Wireshark uses **3 logical operators**:

- AND
 - OR
 - NOT
-

1 ☐ AND Filter (&&)

☞ **Meaning: Both conditions must be true**

◆ Syntax

`condition1 && condition2`

◆ Example 1: HTTP traffic from one IP

`ip.addr == 192.168.1.10 && http`

✓ Shows packets that are:

- From/to 192.168.1.10
 - AND are HTTP packets
-

◆ Example 2: SSH traffic to a server

`tcp.port == 22 && ip.dst == 192.168.1.20`

✓ Shows SSH packets going **to** that server

2 □ OR Filter (||)

☞ **Meaning:** Any one condition can be true

◆ Syntax

```
condition1 || condition2
```

◆ Example 1: HTTP or HTTPS traffic

```
tcp.port == 80 || tcp.port == 443
```

✓ Shows:

- HTTP **OR**
 - HTTPS traffic
-

◆ Example 2: TCP or UDP traffic

```
tcp || udp
```

✓ Shows packets using **TCP or UDP**

3 □ NOT Filter (!)

☞ **Meaning:** Exclude something

◆ Syntax

```
!condition
```

◆ Example 1: Show everything except DNS

```
!dns
```

✓ Hides DNS packets

◆ Example 2: Hide traffic from one IP

```
!ip.addr == 192.168.1.5
```

✓ Shows packets **except** from that IP

4□ Using Brackets (Very Important 🔥)

Brackets help Wireshark understand **priority**.

◆ Example: HTTP or HTTPS from one IP

```
ip.addr == 192.168.1.10 && (tcp.port == 80 || tcp.port == 443)
```

✦ Means:

- Traffic from 192.168.1.10
 - AND (HTTP **or** HTTPS)
-

5□ Real-Life Simple Examples (CTF / Practice)

🔒 Find login requests (HTTP only)

```
http.request.method == "POST" && ip.addr == 192.168.1.10
```

✂ Detect SSH brute-force behavior

```
tcp.port == 22 && tcp.flags.syn == 1
```

🚫 Remove noisy traffic

```
!(arp || dns)
```

One-Line Summary (Remember This)

Operator	Meaning	Example
&&	AND	ip.addr==X && http
!	NOT	!dns